

Smart Farm Management System

Project Report

OOP II Module — Project Report

Errors & Debugging · UML Diagrams · Code Snippets

Module: Object-Oriented Programming II

Hardware: ESP32 + DHT11 Sensor + FC-28 + R307

Stack: Java, MySQL, JavaFX, Arduino C++

Year: 2025 / 2026

Mostafa Mahmoud Aborehab	241008538
Jouiria Mohammed Hassan	241008818
Ahmed Hagag Abdelhamed	241002485
Mohamed Ahmed Mahmoud Abdelbary	241009209
Mariam Yasser Dahab	241008888

Contents

1	Project Information	2
1.1	Group Members	2
1.2	Selected Business Domain	2
1.3	Steps to Run the Project	2
1.4	Database Setup	3
1.5	Socket Port Number	3
2	Errors & Debugging	4
3	UML Diagrams	5
4	Code Snippets	6
4.1	Class, Constructor & Methods	6
4.2	JavaFX Stage (Programmatic — No FXML)	7
4.3	JDBC Database Programming	9
4.3.1	Database Connection (Singleton)	9
4.3.2	CRUD with PreparedStatement	10
4.4	Multithreading	12
4.4.1	Starting the Server on a Daemon Thread	12
4.4.2	Thread Pool for Multiple Devices	12
4.5	Socket Programming	13
4.6	Generics	14
4.6.1	Generic Interface Definition	15
4.6.2	Concrete Implementation	15

1. Project Information

1.1 Group Members

Name	ID	Role
Mostafa Mahmoud Aborehab	241008538	Team Lead, Backend & IoT
Jouiria Mohammed Hassan	241008818	Frontend & UI
Ahmed Hagag Abdelhamed	241002485	Android & Integration
Mohamed Ahmed Mahmoud Abdelbary	241009209	Backend & Database
Mariam Yasser Dahab	241008888	UI/UX & Testing

Table 1.1: Group members and IDs

1.2 Selected Business Domain

Agriculture / Smart Farming. The system manages a real farm operation — monitoring field conditions with IoT sensors, tracking crops and plots, assigning workers to tasks, logging harvests, and generating reports. The domain was chosen because it naturally requires every OOP II concept: multithreading (concurrent sensor data), networking (TCP sockets), database (JDBC/MySQL), GUI (JavaFX), and file I/O (CSV export).

1.3 Steps to Run the Project

1. **Prerequisites:** Install Java 17+, Maven, and MySQL 8.

2. **Clone the repository:**

```
git clone https://github.com/GoBloxy/Agrilliant.git
```

3. **Configure the database** (see Section 1.4 below).

4. **Build and run:**

```
cd Agrilliant
mvn clean javafx:run
```

5. The application opens the sign-in screen. Log in with your credentials or use fingerprint scan for worker access.

6. The TCP sensor server starts automatically on port **8080** (see Section 1.5).

1.4 Database Setup

The project uses a **MySQL 8** database. Connection details are stored in `src/main/resources/db.properties`

```
db.url=jdbc:mysql://<host>:3306/agrilliant
db.user=<username>
db.password=<password>
```

Setup steps:

1. Create a MySQL database named **agrilliant**.
2. The application uses a cloud-hosted MySQL instance by default. To use a local database, update `db.properties` with your local MySQL credentials.
3. Tables are created automatically by the application on first run, or you can run the schema script from the project documentation.

1.5 Socket Port Number

The TCP sensor server listens on port **8080**. ESP32 devices connect to this port to send live sensor data.

Service	Port	Description
TCP Sensor Server	8080	Receives live data from ESP32 devices
MQTT Broker	1883	Distributes sensor data to all clients (HiveMQ public broker)
MySQL Database	3306	Persistent storage for all farm data

Table 1.2: Network ports used by the system

Important

Port 8080 must be available on the machine running the server. If another application is using this port, the TCP server will print a warning but the application will still function — MQTT will deliver sensor data independently.

2. Errors & Debugging

This section will document errors encountered during development, their root causes, and how they were resolved.

3. UML Diagrams

This section will contain the UML class diagram for the system.

4. Code Snippets

This chapter presents representative code snippets from the Smart Farm system, each demonstrating a core OOP II concept.

4.1 Class, Constructor & Methods

The `Alert` class models a farm alert triggered by sensor readings. It demonstrates:

- An inner enum (`Severity`) to restrict field values
- Multiple constructors (overloading) for different creation contexts
- Instance methods that encapsulate behaviour (`resolve()`, `isCritical()`)
- Encapsulation with private fields and public getters/setters

```
1 public class Alert {
2     public enum Severity { INFO, WARNING, CRITICAL }
3
4     private int          alertId;
5     private String       alertType;
6     private Severity     severity;
7     private String       message;
8     private boolean      resolved;
9     private LocalDateTime timestamp;
10    private int          plotId;
11
12    // Full constructor (used when loading from database)
13    public Alert(int alertId, String alertType, Severity severity
14        ,
15                String message, boolean resolved,
16                LocalDateTime timestamp, int plotId) {
17        this.alertId    = alertId;
18        this.alertType  = alertType;
19        this.severity   = severity;
20        this.message    = message;
21        this.resolved   = resolved;
22        this.timestamp  = timestamp;
23        this.plotId     = plotId;
24    }
25
26    // Convenience constructor (used when creating a new alert)
27    public Alert(String alertType, Severity severity,
28                String message, int plotId) {
29        this.alertId    = -1;
30        this.alertType  = alertType;
31        this.severity   = severity;
32        this.message    = message;
33        this.resolved   = false;
34        this.timestamp  = LocalDateTime.now();
35        this.plotId     = plotId;
36    }
37 }
```

```
37      // Behaviour methods
38      public void      resolve()      { this.resolved = true; }
39      public boolean isCritical() { return severity == Severity.
        CRITICAL; }
40      public boolean isResolved() { return resolved; }
41
42      // Getters
43      public int      getAlertId()    { return alertId; }
44      public String   getAlertType() { return alertType; }
45      public Severity getSeverity()   { return severity; }
46      public String   getMessage()   { return message; }
47      public LocalDateTime getTimestamp() { return timestamp; }
48      public int      getPlotId()    { return plotId; }
49
50      @Override
51      public String toString() {
52          return "[" + severity + "] " + alertType
53              + " -- " + message + " (plot=" + plotId + ")";
54      }
55  }
```

Listing 4.1: Alert — class with enum, constructors, and methods

Key OOP Concepts

- **Encapsulation** — all fields are **private**; access is through methods.
- **Constructor overloading** — two constructors serve different use cases (DB load vs. new alert).
- **Enum** — **Severity** restricts the value to exactly three options, enforced at compile time.

4.2 JavaFX Stage (Programmatic — No FXML)

The application entry point creates the **Stage**, configures the window, and launches background services. This is pure Java — no FXML layout file is used for the Stage itself.

```
1  public class Main extends Application {
2
3      @Override
4      public void start(Stage primaryStage) throws Exception {
5          // Build the root layout in Java code
6          Parent root = FXMLLoader.load(
7              getClass().getResource("/fxml/signin.fxml"));
8
9          // Create Scene and apply stylesheet
10         Scene scene = new Scene(root, 1000, 650);
11         scene.getStylesheets().add(
12             getClass().getResource("/css/farm-theme.css")
13         );
14     }
15 }
```



```
13         .toExternalForm());
14
15         // Configure the Stage (window)
16         primaryStage.setTitle(
17             "Agrilliant -- Smart Farm Management System");
18         primaryStage.getIcons().add(new Image(
19             getClass().getResourceAsStream("/images/logo-dark.png")
20             ));
21         primaryStage.setScene(scene);
22         primaryStage.setMinWidth(1300);
23         primaryStage.setMinHeight(820);
24         primaryStage.setMaximized(true);
25         primaryStage.setOnCloseRequest(e -> System.exit(0));
26         primaryStage.show();
27
28     public static void main(String[] args) {
29         launch(args);
30     }
31 }
```

Listing 4.2: Main — JavaFX Application with programmatic Stage setup

The `SettingsPage` is a screen built **entirely without FXML** — all UI controls are created and laid out in pure Java:

```
1 public class SettingsPage extends VBox {
2
3     private final TextField txtServerIp    = new TextField("
4         192.168.8.141");
5     private final TextField txtServerPort = new TextField("8080")
6         ;
7     private final Spinner<Integer> spnInterval = new Spinner<>(5,
8         300, 60, 5);
9     private final ComboBox<String> cmbUnit    = new ComboBox<>()
10        ;
11
12     public SettingsPage() {
13         setSpacing(24);
14         setPadding(new Insets(32, 40, 32, 40));
15
16         // Header
17         Label title = new Label("Settings");
18         title.setFont(Font.font("System", FontWeight.BOLD, 24));
19
20         // Server section
21         GridPane serverGrid = new GridPane();
22         serverGrid.setHgap(16);
23         serverGrid.setVgap(12);
24         serverGrid.add(new Label("Server IP"), 0, 0);
25         serverGrid.add(txtServerIp, 1, 0);
```

```
22         serverGrid.add(new Label("Port"), 2, 0);
23         serverGrid.add(txtServerPort, 3, 0);
24
25         // Temperature unit selector
26         cmbUnit.getItems().addAll("Celsius", "Fahrenheit");
27         cmbUnit.setValue("Celsius");
28
29         // Save button with event handler
30         Button btnSave = new Button("Save Changes");
31         btnSave.setOnAction(e -> {
32             Alert alert = new Alert(Alert.AlertType.INFORMATION,
33                 "Settings saved.", ButtonType.OK);
34             alert.showAndWait();
35         });
36
37         // Assemble the layout
38         getChildren().addAll(title, serverGrid, cmbUnit, btnSave);
39     }
40 }
```

Listing 4.3: SettingsPage — programmatic JavaFX UI (no FXML)

Why Two Approaches?

JavaFX supports two UI-building strategies:

- **FXML** — XML layout files (used for complex screens like the dashboard)
- **Programmatic** — pure Java code (used for simpler pages like Settings)

The **Stage** and **Scene** are always created in Java code. FXML only defines the *content* inside the scene.

4.3 JDBC Database Programming

4.3.1 Database Connection (Singleton)

The `DBConnection` class provides a single shared `Connection` instance using the Singleton pattern. Credentials are loaded from a properties file.

```
1 public class DBConnection {
2     private static Connection connection;
3     private static String url, user, password;
4
5     private DBConnection() {} // prevent instantiation
6
7     static {
8         // Load credentials from db.properties
9         try (InputStream input = DBConnection.class
10             .getClassLoader()
```

```
11         .getResourceAsStream("db.properties")) {
12             Properties props = new Properties();
13             props.load(input);
14             url = props.getProperty("db.url");
15             user = props.getProperty("db.user");
16             password = props.getProperty("db.password");
17         } catch (IOException e) {
18             System.err.println("Failed to load db.properties");
19         }
20     }
21
22     public static Connection getInstance() {
23         try {
24             if (connection == null || connection.isClosed()) {
25                 connection = DriverManager.getConnection(
26                     url, user, password);
27             }
28         } catch (SQLException e) {
29             System.err.println("DB connection failed: "
30                 + e.getMessage());
31         }
32         return connection;
33     }
34 }
```

Listing 4.4: DBConnection — Singleton JDBC connection

4.3.2 CRUD with PreparedStatement

The WorkerDAO demonstrates JDBC CRUD operations using PreparedStatement to prevent SQL injection:

```
1 public class WorkerDAO implements GenericDAO<Worker> {
2     private final Connection conn = DBConnection.getInstance();
3
4     @Override
5     public void save(Worker item) throws SQLException {
6         String sql = "INSERT INTO worker (full_name, phone, email
7             , "
8             + "job_title, on_duty) VALUES (?, ?, ?, ?, ?)"
9             ;
10         try (PreparedStatement ps = conn.prepareStatement(sql,
11             Statement.RETURN_GENERATED_KEYS)) {
12             ps.setString(1, item.getFullName());
13             ps.setString(2, item.getPhone());
14             ps.setString(3, item.getEmail());
15             ps.setString(4, item.getJobTitle());
16             ps.setBoolean(5, item.isOnDuty());
17             ps.executeUpdate();
18         }
```

```
17         // Retrieve auto-generated primary key
18         try (ResultSet keys = ps.getGeneratedKeys()) {
19             if (keys.next())
20                 item.setWorkerId(keys.getInt(1));
21         }
22     }
23 }
24
25 @Override
26 public List<Worker> getAll() throws SQLException {
27     String sql = "SELECT * FROM worker ORDER BY full_name";
28     List<Worker> workers = new ArrayList<>();
29     try (PreparedStatement ps = conn.prepareStatement(sql);
30         ResultSet rs = ps.executeQuery()) {
31         while (rs.next()) {
32             workers.add(new Worker(
33                 rs.getInt("worker_id"),
34                 rs.getString("full_name"),
35                 rs.getString("phone"),
36                 rs.getString("email"),
37                 rs.getString("job_title"),
38                 rs.getBoolean("on_duty")
39             ));
40         }
41     }
42     return workers;
43 }
44
45 @Override
46 public void delete(int id) throws SQLException {
47     String sql = "DELETE FROM worker WHERE worker_id = ?";
48     try (PreparedStatement ps = conn.prepareStatement(sql)) {
49         ps.setInt(1, id);
50         ps.executeUpdate();
51     }
52 }
53 }
```

Listing 4.5: WorkerDAO — INSERT and SELECT with PreparedStatement

JDBC Best Practices Shown

- **PreparedStatement** — prevents SQL injection by using ? placeholders instead of string concatenation.
- **try-with-resources** — automatically closes PreparedStatement and ResultSet, even if an exception is thrown.
- **Generated keys** — retrieves the auto-incremented ID after an INSERT.

4.4 Multithreading

The system uses multithreading in two ways: a **daemon thread** for the TCP server and an **ExecutorService thread pool** that spawns a new thread for each connected sensor device.

4.4.1 Starting the Server on a Daemon Thread

The TCP server blocks forever waiting for connections, so it must run on a background thread. A **daemon thread** is used so it automatically stops when the application closes.

```
1 // Inside Main.start() -- after showing the JavaFX stage:
2
3 // Start TCP sensor server on a daemon thread
4 Thread serverThread = new Thread(() -> new FarmServer().start());
5 serverThread.setDaemon(true);           // dies when app closes
6 serverThread.setName("FarmServer-TCP");
7 serverThread.start();
```

Listing 4.6: Main — launching the server on a daemon thread

4.4.2 Thread Pool for Multiple Devices

The `FarmServer` uses a `CachedThreadPool` to handle multiple ESP32 devices simultaneously. Each device gets its own `SensorHandler` thread.

```
1 public class FarmServer {
2     private static final int PORT = 8080;
3
4     public void start() {
5         ExecutorService threadPool = Executors.
6             newCachedThreadPool();
7         try (ServerSocket serverSocket = new ServerSocket(PORT))
8         {
9             System.out.println("Farm Server started on port " +
10                 PORT);
11
12             while (true) {
13                 Socket deviceSocket = serverSocket.accept();
14                 System.out.println("Device connected: "
15                     + deviceSocket.getInetAddress());
16                 // Each device runs on its own thread
17                 threadPool.submit(new SensorHandler(deviceSocket));
18             }
19         } catch (IOException e) {
20             System.err.println("Server error: " + e.getMessage());
21         }
22     }
23 }
```

```
19     }
20 }
```

Listing 4.7: FarmServer — thread pool accepts multiple device connections

Multithreading Concepts Demonstrated

- **Daemon thread** — background thread that does not prevent JVM shutdown.
- **ExecutorService** — manages a pool of threads; avoids creating threads manually for each connection.
- **Runnable** — `SensorHandler` implements `Runnable`, allowing it to be submitted to the thread pool.
- **Concurrency** — multiple ESP32 sensors can connect and send data simultaneously without blocking each other.

4.5 Socket Programming

The system uses TCP sockets to receive live data from ESP32 devices. The **server side** uses `ServerSocket` to listen for connections, and the **handler** uses `Socket.getInputStream()` to read data line by line.

```
1  public class SensorHandler implements Runnable {
2      private final Socket socket;
3      private final SensorService sensorService = new SensorService
4          ();
5
6      public SensorHandler(Socket socket) {
7          this.socket = socket;
8      }
9
10     @Override
11     public void run() {
12         try (BufferedReader reader = new BufferedReader(
13             new InputStreamReader(socket.getInputStream())))
14         {
15             String line;
16             while ((line = reader.readLine()) != null) {
17                 SensorReading reading = parseLine(line);
18                 if (reading != null) {
19                     sensorService.processReading(reading);
20                 }
21             }
22         } catch (IOException e) {
23             System.out.println("Device disconnected: "
24                 + socket.getInetAddress());
25         }
26     }
27 }
```

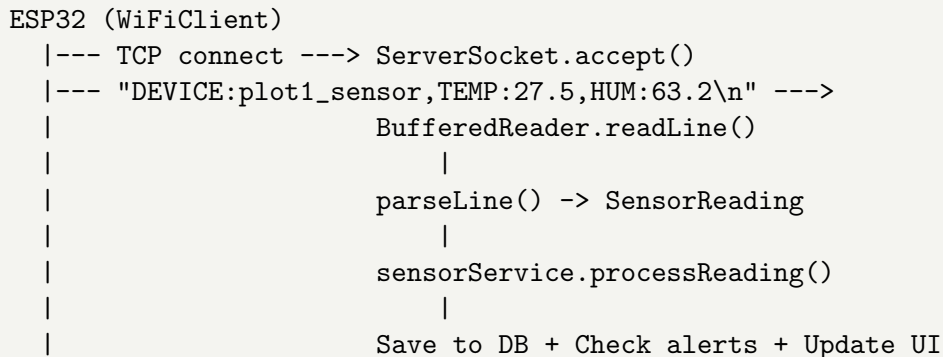
```

24 }
25
26 // Parses "DEVICE:plot1_sensor,TEMP:27.50,HUM:63.20"
27 private SensorReading parseLine(String raw) {
28     try {
29         String[] parts = raw.split(",");
30         String device = parts[0].split(":")[1];
31         float temp    = Float.parseFloat(parts[1].split(":")
32             [1]);
33         float hum      = Float.parseFloat(parts[2].split(":")
34             [1]);
35         return new SensorReading(device, temp, hum,
36             LocalDateTime.now());
37     } catch (Exception e) {
38         System.err.println("Bad data format: " + raw);
39         return null;
40     }
41 }

```

Listing 4.8: SensorHandler — reads data from a TCP socket

The data flow is:



Socket Programming Concepts

- **ServerSocket** — listens on a port and accepts incoming connections.
- **Socket** — represents one end of a two-way TCP connection.
- **InputStream wrapping** — `Socket` → `InputStreamReader` → `BufferedReader` for line-by-line reading.
- **Protocol design** — a simple text-based protocol (`KEY:VALUE,KEY:VALUE`) parsed with `split()`.

4.6 Generics

The system uses a **generic DAO interface** to define standard CRUD operations for *any* model class. Each concrete DAO implements this interface with a specific type.

4.6.1 Generic Interface Definition

```
1 public interface GenericDAO<T> {
2     void save(T item) throws SQLException;
3     T getById(int id) throws SQLException;
4     List<T> getAll() throws SQLException;
5     void update(T item) throws SQLException;
6     void delete(int id) throws SQLException;
7 }
```

Listing 4.9: GenericDAO<T> — generic interface for all data access objects

4.6.2 Concrete Implementation

Each model class gets its own DAO that implements GenericDAO with the appropriate type parameter:

```
1 public class TaskDAO implements GenericDAO<Task> {
2     private final Connection conn = DBConnection.getInstance();
3
4     @Override
5     public void save(Task item) throws SQLException {
6         String sql = "INSERT INTO tasks (description, status, "
7             + "due_date, plot_id) VALUES (?, ?, ?, ?)";
8         try (PreparedStatement ps = conn.prepareStatement(sql,
9             Statement.RETURN_GENERATED_KEYS)) {
10             ps.setString(1, item.getDescription());
11             ps.setString(2, item.getStatus().name());
12             ps.setDate(3, Date.valueOf(item.getDueDate()));
13             ps.setInt(4, item.getPlotId());
14             ps.executeUpdate();
15
16             try (ResultSet keys = ps.getGeneratedKeys()) {
17                 if (keys.next())
18                     item.setTaskId(keys.getInt(1));
19             }
20         }
21     }
22
23     @Override
24     public List<Task> getAll() throws SQLException {
25         List<Task> tasks = new ArrayList<>();
26         String sql = "SELECT * FROM tasks ORDER BY due_date";
27         try (PreparedStatement ps = conn.prepareStatement(sql);
28             ResultSet rs = ps.executeQuery()) {
29             while (rs.next()) {
30                 tasks.add(new Task(
31                     rs.getInt("task_id"),
32                     rs.getString("description"),
33                     Task.Status.valueOf(rs.getString("status")),
```



```

34         rs.getDate("due_date").toLocalDate(),
35         rs.getInt("plot_id")
36     ));
37     }
38 }
39 return tasks;
40 }
41
42 @Override
43 public void delete(int id) throws SQLException {
44     String sql = "DELETE FROM tasks WHERE task_id = ?";
45     try (PreparedStatement ps = conn.prepareStatement(sql)) {
46         ps.setInt(1, id);
47         ps.executeUpdate();
48     }
49 }
50
51 // getById() and update() follow the same pattern...
52 }

```

Listing 4.10: TaskDAO — implements GenericDAO<Task>

All DAO classes in the project implement this single generic interface:

DAO Class	Implements	Model
TaskDAO	GenericDAO<Task>	Farm tasks assigned to workers
WorkerDAO	GenericDAO<Worker>	Farm workers
AlertDAO	GenericDAO<Alert>	Sensor-triggered alerts
CropDAO	GenericDAO<Crop>	Planted crops
PlotDAO	GenericDAO<Plot>	Farm plots of land
DeviceDAO	GenericDAO<Device>	IoT sensor devices

Table 4.1: All DAO classes implement the same generic interface

Generics Concepts Demonstrated

- **Type parameter** <T> — the interface is written once and works for any model type.
- **Type safety** — GenericDAO<Task> guarantees that `save()` only accepts a `Task`, not a `Worker` or `Alert`. The compiler catches type mismatches.
- **Code reuse** — the contract (five CRUD methods) is defined once; six classes implement it without duplicating the interface.
- **Bounded usage** — `List<T>` in the return type of `getAll()` is itself a generic, showing generics composed inside generics.